

1 - Couicable

fonctions simples, docstring, retour de plusieurs valeurs

1. Écrire la fonction `somme_chiffres()` qui retourne la somme des chiffres composant le nombre passé en paramètre.
2. Écrire la fonction `nombre_chiffres()` qui retourne le nombre des chiffres composant le nombre passé en paramètre.
3. Écrire la fonction `separe_nombre()` qui retourne les parties droite et gauche après avoir séparé le nombre passé en paramètre en 2 parties de taille égale.
4. Écrire le programme principal qui demande un nombre à l'utilisateur et affiche si celui ci est couicable ou non. Un nombre est dit couicable s'il est composé d'un nombre de chiffres pair et si la sommes des chiffres de sa partie droite est égale à la somme des chiffres de sa partie gauche.
5. Écrire la fonction `somme_chiffres()` mais cette fois-ci de facon récursive.

correction

```
# correction qui manipule les entiers

def somme_chiffres_it(nombre: int) -> int:
    '''Retourne la somme des chiffres de l'entier passé en paramètre, version itérative'''
    s = 0
    while (nombre != 0):
        s += nombre % 10
        nombre //= 10
    return s

def somme_chiffres_rec(nombre: int) -> int:
    '''Retourne la somme des chiffres de l'entier passé en paramètre, version récursive'''
    if nombre == 0:
        return 0
    else :
        return nombre % 10 + somme_chiffres_rec(nombre // 10)

def nombre_chiffres(nombre: int) -> int:
    '''Retourne le nombre de chiffres de l'entier passé en paramètre'''
    n = 0
    while (nombre != 0):
        nombre //= 10
        n += 1
    return n

def separe_nombre(nombre: int) -> (int,int):
    '''Sépare en deux parties égales le nombre passé en paramètre'''
    n = nombre_chiffres(nombre)
    pdroite = nombre % (10 ** (n // 2))
    pgauche = nombre // (10 ** (n // 2))
    return pgauche, pdroite
```

```
def main():
    ''' La gestion des exceptions est optionnelle, selon l'avancement du cours'''
    try :
        nombre = int(input("Entrer un nombre : "))
        if nombre_chiffres(nombre)%2 != 0:
            print("Non couicable : nombre de chiffres impair")
        else :
            gauche, droit = separe_nombre(nombre)
            if (gauche, droit) != (0, 0) \
            and somme_chiffres_it(gauche) == somme_chiffres_it(droit):
                print("Couicable")
            else:
                print("Non couicable")
    except ValueError:
        print("Ce n'est pas un entier positif")
        exit(-1)

if __name__ == "__main__":
    main()
```

correction

```
# Correction qui transforme les entiers en chaines de caractères pour faciliter les
traitements

def somme_chiffres(nombre: int) -> int:
    '''Retourne la somme des chiffres de l'entier passé en paramètre'''
    return sum(int(chiffre) for chiffre in str(nombre))

def nombre_chiffres(nombre: int) -> int:
    '''Retourne le nombre de chiffres de l'entier passé en paramètre'''
    return len(str(nombre))

def nb_chiffre_pair(nombre : int) -> int :
    '''Retourne vra i si le nombre de chiffres de l'entier passé en paramètre est
    pair, faux sinon'''
    return nombre_chiffres(nombre) % 2 == 0

def separe_nombre(nombre: int) -> (int,int):
    '''Sépare en deux parties égales le nombre passé en paramètre'''
    milieu = nombre_chiffres(nombre) // 2
    nombre_str=str(nombre)
    return int(nombre_str[0: milieu]), int(nombre_str[milieu: len(nombre_str)])

def main():
    nombre = int(input("Entrer un nombre : "))
    if not nb_chiffre_pair(nombre) :
        print("Non couicable : nombre de chiffres impair")
    else :
        gauche, droit = separe_nombre(nombre)
        if somme_chiffres(gauche) == somme_chiffres(droit) :
            print("Couicable")
        else:
```

```
print("Non couicable")

if __name__ == "__main__":
    main()
```

2 - Carré magique

fonctions simples, docstring, modules, nombre d'arguments variable, liste de liste

Nous voudrions définir un module `carre_magique` qui contient les fonctions relatives aux carrés magiques. Les fonctions demandées dans les questions 1 à 6 seront définies dans ce module. La question 7 sera définie dans un fichier contenant aussi la fonction `main()` qui utilisera ce module.

Définition d'un carré magique

On considère un entier n strictement positif. Un carré magique d'ordre n est une matrice carrée d'ordre n (n lignes et n colonnes), qui contient des nombres entiers strictement positifs. Ces nombres sont disposés de sorte que les sommes sur chaque ligne, les sommes sur chaque colonne et les sommes sur chaque diagonale principale soient égales. La valeur de ces sommes est appelée : constante magique.

Exemple : Carré magique d'ordre 3, sa constante magique vaut 45

21	7	17	→ 45
11	15	19	→ 45
13	23	9	→ 45

↙

↓

↓

↓

↘

45

45

45

45

45

1. Écrire la fonction `somme_ligne(mat, i)`, qui reçoit en paramètres une matrice carrée `mat` contenant des nombres, et un entier `i` qui représente l'indice d'une ligne dans `mat`. La fonction retourne la somme des nombres de la ligne d'indice `i` dans `mat`.
2. Écrire la fonction `somme_colonne(mat, j)`, qui reçoit en paramètres une matrice carrée `mat` contenant des nombres, et un entier `j` qui représente l'indice d'une colonne dans `mat`. La fonction retourne la somme des nombres de la colonne `j` dans `mat`.
3. Écrire la fonction `somme_diag1(mat)`, qui reçoit en paramètre une matrice carrée `mat` contenant des nombres, et qui retourne la somme des éléments de la première diagonale principale dans `mat`.
4. Écrire la fonction `somme_diag2(mat)`, qui reçoit en paramètre une matrice carrée `mat` contenant des nombres, et qui retourne la somme des éléments de la deuxième diagonale principale dans `mat`.
5. Écrire la fonction `magique(mat_c)`, qui reçoit en paramètre une matrice carrée `mat_c` contenant des entiers strictement positifs, et qui retourne `True`, si la matrice `mat_c` est un carré magique et `False`, sinon.

Un carré magique normal d'ordre n est un carré magique d'ordre n , constitué de tous les nombres entiers positifs compris entre 1 et n^2 .

Exemple : Carré magique normal d'ordre 4, composé des nombres entiers : 1, 2, 3, ..., 15, 16.

4	14	15	1
9	7	6	12
5	11	10	8
16	2	3	13

6. Écrire la fonction `carre_magique_normal(mat_c)`, qui reçoit en paramètre une matrice carrée `mat_c` qui représente un carré magique. La fonction retourne `True` si le carré magique `mat_c` est normal, sinon, elle retourne `False`. Par exemple,

```
carre_magique_normal([[8, 1, 6], [3, 5, 7], [4, 9, 2]]) -> True
carre_magique_normal([[21, 7, 17], [11, 15, 19], [13, 23, 9]]) -> False
```

correction

```
# Version exploitant les compréhensions de listes et les fonction/méthode sum(),
# sort() ...

# Création d'un alias de type
Matrice = list[list[int]]

def somme_ligne(mat: Matrice, i: int) -> int:
    '''retourne la somme des composantes d'une ligne d'une matrice'''
    return sum(mat[i])

def somme_colonne(mat: Matrice, j: int) -> int:
    '''retourne la somme des composantes d'une colonne d'une matrice'''
    return sum([line[j] for line in mat])

def somme_diag_1(mat: Matrice) -> int:
    '''retourne la somme des composantes de la diagonale hg -> bd d'une matrice
    carrée'''
    return sum(line[i][i] for i in range(len(mat)))

def somme_diag_2(mat: Matrice) -> int:
    '''retourne la somme des composantes de la diagonale hd -> bg d'une matrice
    carrée'''
    return sum(line[i][-i-1] for i in range(len(mat)))

def magique(mat: Matrice) -> bool:
    '''détermine si une matrice carrée composée d'entier est magique ou non'''
    res = somme_ligne(mat, 0)
    if somme_diag_1(mat) != res or somme_diag_2(mat) != res:
        return False
    for i in range(len(mat)):
        if somme_ligne(mat, i) != res or somme_colonne(mat, i) != res:
            return False
    return True

def magique_2(mat: Matrice) -> bool:
    return len(set([somme_ligne(mat, i) for i in range(len(mat))] +
                  [somme_colonne(mat, i) for i in range(len(mat))])) == len(mat)
```

```

        [somme_diag_1(mat), somme_diag_2(mat))]) == 1

def carre_magique_normal(mat: list[list[int]]) -> bool:
    '''détermine si un carré magique est normal ou non '''
    nombres_dans_carre = [nombre for line in mat for nombre in line]
    nombres_dans_carre.sort()
    # ou      nombres_dans_carre = sorted(nombre for line in mat for nombre in line)
    nb_ordre=[i+1 for i in range(len(mat) ** 2)]
    return nombres_dans_carre == nb_ordre

```

7. Dans une autre fichier, écrire une fonction `affiche_test_cm(*args)` qui reçoit comme paramètres une ou plusieurs matrices qui les teste et affiche si elles sont magiques et le cas échéant si elles sont “normales”. Écrire le programme principal qui permet de tester les exemples ci-dessous.

```

C1 = [[8, 1, 6], [3, 5, 7], [4, 9, 2]]
C2 = [[21, 7, 17], [11, 15, 19], [13, 23, 9]]
C3 = [
    [30, 39, 48, 1, 10, 19, 28],
    [38, 47, 7, 9, 18, 27, 29],
    [46, 6, 8, 17, 26, 35, 37],
    [5, 14, 16, 25, 34, 36, 45],
    [13, 15, 24, 33, 42, 44, 4],
    [21, 23, 32, 41, 43, 3, 12],
    [22, 31, 40, 49, 2, 11, 20],
]

```

correction

```

import ex2 as cm

def affiche_test_cm(*args):
    '''appelle les fonctions de test de carré magique et normal pour les matrices
    contenues dans la liste passée en paramètre.'''
    for C in args:
        if cm.magique(C):
            if cm.carre_magique_normal(C):
                print("Carré magique normal de valeur", cm.somme_ligne(C, 0))
            else:
                print("Carré magique de valeur", cm.somme_ligne(C, 0))
        else:
            print("Ce n'est pas un carré magique")

def main():
    C1 = [[8, 1, 6], [3, 5, 7], [4, 9, 2]]
    C2 = [[21, 7, 17], [11, 15, 19], [13, 23, 9]]
    C3 = [
        [30, 39, 48, 1, 10, 19, 28],
        [38, 47, 7, 9, 18, 27, 29],
        [46, 6, 8, 17, 26, 35, 37],
        [5, 14, 16, 25, 34, 36, 45],
        [13, 15, 24, 33, 42, 44, 4],
        [21, 23, 32, 41, 43, 3, 12],
        [22, 31, 40, 49, 2, 11, 20],
    ]
    C4 = [[8, 1, 6], [3, 5, 7], [4, 12, 2]]

```

```
affiche_test_cm(C1,C2,C3,C4)

if __name__ == "__main__":
    main()
```

3 - Décorateur et exception

fonctions avec decorateur, exceptions

On voudrait reprendre l'exercice 1 et le rendre plus robuste vis à vis de la saisie du nombre par l'utilisateur.

1. Ecrire un décorateur qui s'appelle `@nombre` et qui transforme le paramètre en nombre si c'est une chaîne de caractères qui représente un nombre. Si ce n'est pas un nombre ou une chaîne qui représente un nombre, on applique la fonction sur 0. On appliquera ce décorateur aux fonctions de l'exercice 1.
2. Modifier le code du décorateur pour qu'il déclenche une exception si le paramètre n'est ni un nombre ni une chaîne représentant un nombre

correction

```
# question 1
def nombre1(function):
    ''' decorateur visant à rendre plus robuste les e/s'''
    def check_int(x):
        if type(x)==str:
            for c in x:
                if c not in "0123456789" :
                    return function(0)
            return function(int(x))
        elif type(x)==int :
            return function(x)
        else :
            return function(0)
    return check_int

# question 2
def nombre2(function):
    ''' decorateur visant à rendre plus robuste les e/s'''
    def wrapper(x):
        if type(x)==str:
            for c in x:
                if c not in "0123456789" :
                    raise TypeError("Le paramètre est une chaîne mais ne contient pas un nombre")
            return function(int(x))
        elif type(x)==int :
            return function(x)
        else :
            raise TypeError("Le paramètre n'est pas interpretable comme un nombre entier")
    return wrapper
```

```

@nombre1
def somme_chiffres(nombre: int) -> int:
    return sum(int(chiffre) for chiffre in str(nombre))

@nombre1
def somme_chiffres_rec(nombre: int) -> int:
    if nombre == 0:
        return 0
    else:
        return nombre % 10 + somme_chiffres_rec(nombre // 10)

@nombre1
def nombre_chiffres(nombre: int) -> int:
    return len(str(nombre))

@nombre1
def separe_nombre(nombre: int) -> tuple:
    milieu = nombre_chiffres(nombre) // 2
    nombre_str = str(nombre)
    return int(nombre_str[0: milieu]), int(nombre_str[milieu: len(nombre_str)])

@nombre2
def test(x):
    print('on applique le decorateur à la fonction de test avec le paramètre ', x)
    return x

def main():
    try:

        print(test(12))
        print(test("24"))
        # print(test(123.5)). enlever le commentaire pour tester le décorateur et
        # les exceptions

        nombre = int(input("Entrer un nombre : "))
        print(type(nombre))
        if nombre_chiffres(nombre) % 2 != 0:
            print("Non couicable, nb de chiffres impair")
        else:
            gauche, droit = separe_nombre(nombre)
            if (gauche, droit) != (0, 0) and somme_chiffres(gauche) ==
                somme_chiffres(droit):
                print("Couicable")
            else:
                print("Non couicable")
    except TypeError as e:
        print(e)
    except ValueError:
        print("erreur de valeur")

if __name__ == "__main__":
    main()

```